TEMCO LEARN

Temco Platform

`temco-ai` · built Nov 10, 2026

`temco-ai`

The local-AI module: it runs a model on this machine and answers from the documentation it is given.

Folder	`extensions/temco-ai`
Extension id	`temcocontrols.temco-ai`
Namespace	`temcoAi`
State	active
Container	**Temco AI** (`temcoAi`) in the right side bar — the panel (`temcoAi.panel`)
Requires	`temcocontrols.temco-docs`
Settings	`temcoAi.modelFolder` , `temcoAi.allowUnpinned` , `temcoAi.runtime` , `temcoAi.model` , `temcoAi.idleMinutes`

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the panel, and the model the rest of the platform reads (`status` , `facts` , `modelLabel` , `analyse` , `draft`)
`src/docs.ts`	what this module reads of `temco-docs` — the contract in one file, and the only place that crosses modules
`src/surface.ts`	the AI page's view type
`src/aiPage.ts`	the page: the runtime, the model, the folder they live in, the downloads, and the log
`src/ai/catalog.ts`	what can be downloaded: the pinned llama.cpp build with the checksums GitHub publishes, the models on offer, their licences. No network, no filesystem
`src/ai/install.ts`	the model folder, the resumable download and its checksum check, the unpack, and the manifest of what is installed
`src/ai/runtime.ts`	the local server process: started on demand on a loopback port, health-checked, stopped when idle — and the single `complete()` call everything else uses
`src/ai/machine.ts`	what this machine can run, and which catalog entry suits it
`src/ai/history.ts`	the commits a draft is about: `git log` over the page's `code` globs since its last review, metadata and paths only. Never throws
`src/ai/prompts.ts`	loads `prompts/*.md` and fills them in: the house rules, the question, the draft, the area, a new page. The only text that may reach the model
`src/ai/citations.ts`	what an answer pointed at: the ids that resolve, the ids that do not, and the plain term search behind a low-confidence fallback
`src/ai/panel.ts`	the panel: chat about the open page, `Draft update` as a diff, `Ask the area` , and the model line. Decides nothing itself
`src/ai/commands.ts`	the list a person picks from: install, use, remove, open the folder, and the AI's status
`media/`	the panel's script and styles (`ai-panel.js` is a browser asset, not extension code) and the module's icon
`prompts/`	the templates the module sends to the model, one file per task

Commands

Command	What it does
`temcoAi.page`	opens the AI page, or one of its sections (`downloads`)
`temcoAi.openPanel`	focuses the panel in the right side bar
`temcoAi.models`	the list of runtimes and models: what is installed, what can be downloaded, which one is in use
`temcoAi.removeModel`	deletes an installed runtime or model, and clears it from the settings when it was the one in use
`temcoAi.openModelFolder`	opens the folder the downloads live in
`temcoAi.dockRight`	shows the AI in the right side bar
`temcoAi.status`	which runtime and model are in use, whether the server is running, and how to stop it or read its log

How it reads the documentation

One declared dependency, in one direction:

- this module **requires** `temcocontrols.temco-docs` , so that module is activated first and its exports — the sets, `collectPages` , `pageForUri` , the selection, the editor column, one change signal — are ready when this module's `activate()` runs. `src/docs.ts` is where they are declared and where a call reaches them.
- the documentation module **requires nothing** of this one. It calls `temcoAi.page` and `temcoAi.models` for what a person opens, and asks this module's exports for the two things that are work rather than navigation: a body to draft, and a source to analyse. It gates the rows and menu entries it shows on `temcoDocs.aiInstalled` , so an install without this module shows no door that does not open.

The tool server

The assistant can call tools: a C# MCP server lives at `tools/temco-mcp` and this module is its client. The protocol between them, the tools it exposes, how the server is built, and how the application starts and stops it are specified in `design/ai-assistant-and-mcp-server.md` .

What it never does

- Writes a page. A draft goes back to the page that asked for it, and a person accepts it as one undoable edit.

- Leaves the machine. The runtime listens on loopback, models arrive only from a command a person ran, and the only thing that ever leaves is an API read the documentation module asked for.
- Ships native code. A compiled engine is per-OS, so it is downloaded on request into the user's own folder.